Numpy [Num-Numerical, Py = Python].

Why numpy?

→ Numpy is a powerful library in Python used for numerical Computing, ie; Statistical operations, mathematical Computation, probability,

→ Very efficient (Broadcasting)

[fast processing + Memory Consumption is low].

Memory → Primary – RAM (Volatile) – ie; temporary (data lost switch off Computer)

Memory → Secondary – HDrive, SSD.

→ Numpy array's are stored & manipulated in primary memory (RAM), while the program is Running. Data Can be saved to and loaded from Secondary memory (like a hard drive), but actual operations and Computations happen in primary memory.

→ Most powerful numerical processing library in python. Array Oriented Computing is done using Numpy

→ Array – Homogenous, Fixed size, dynamic size.

List – Heterogeneous.

Numpy:- We Can Create an array using Numpy Library.

For installation → Pip install numpy.

importing numpy → import numpy as np → renaming.

range() → inside python inbuilt range function.

np.arange() → inside numpy library, more efficient than python range for range in numpy.

For Calculating time → %%time (it will shows the time utilize to run it).

List → lst = [1,2,3]

O/p:- [1,2,3].

numpy, arr = np.array([1,2,3])

o/p:- [1 2 3].

We didn't get Commas b/w no's in numpy array

→ MS = Millisecond = 1000 M.S = 1 sec.
S = Second = 1.
µS = Microsecond = 1000000 = 1 sec.
nS = nanosec = 1000000000 = 1 sec.

→ 1 Byte = 8 bits (2^3).
1 GB = 1024 MB. (2^{10}).
1 kB = 1024 Bytes.
1 MB = 1024 kB.
1 TB = 1024 GB.

Numpy Array and Its Attributes/Properties.

i. shape;- This attribute returns a tuple indicating the size of each dimension of the array. i.e; rows & Columns

eg:- arr = np.array([[1,2,3],[4,5,6]])
Print (arr. shape).
o/p:- (2,3).
↓ r ↓ c

ii. dtype;- This attribute tells the data type of the elements in the array.
eg:- print (arr. dtype) o/p:- int64.

iii. ndim;- This attribute gives the no. of dimensions (axes) of the array.
eg:- print (arr.ndim) o/p:- 2.

iv. size;- This attribute returns the total no. of elements in the array.
eg:- print (arr. size) o/p:- 6.

v. itemsize:- This attribute returns the size in bytes of each element of the array.
eg:- print (arr.itemsize) o/p:- 8 (bytes).

o/p:- Datatype:- float 64
Itemsize → Integers - 4.
→ Float - 8.

Functions For Creating Numpy Array.

Numpy Provides several functions for Creating arrays with different initial values, shapes and datatypes.

1. np.array:- Convert a Python list or tuple into a Numpy Array

eg:- arr = np.array([1,2,3])
Print(arr) o/p:- [1 2 3].

2. np.zeros:- Create an array filled with zeros.

eg:- a = np.zeros((3,4)) (3 → r, 4 → c)
print(a) o/p:- [[0. 0. 0. 0.]
[0. 0. 0. 0.]
[0. 0. 0. 0.]]

Datatyp:- float64.
Itemsize:- 8.

3. np.ones:- Create an array filled with ones.

eg:- b = np.ones((2,3)) (2 → r, 3 → c)
print(b) op:- [[1. 1. 1.]
[1. 1. 1.]]

Datatype:- float 64.
Itemsize:- 8.

4. np.full:- Create an array filled with a specified value.

eg:- c = np.full((2,2), 7) # Create a 2x2 array filled with 7.
Print(c). (2 → r, 2 → c; 7 → filled with this no)
O/p:- [[7 7 7]
[7 7 7]]. dtype:- int32
itemsize:- 4.

5. np.empty:- Create an uninitialized array. The Content of the array will be random & depends on the state of the memory.

eg:- d = np.empty ((2,3))
Print(d) o/p:- [[2. 2. 2.]
[2. 2. 2.]] dtype:- float 64
itemsize:- 8.

6. np.arange:- Create an array with a range of values.

eg:- e = np.arange(10) → within range 10.
or
np.arrange(1, 10) → within this range.
(1 → start no.)

np.arrange(1, 10, 2).
(1 → start, 10 → end, 2 → step)

o/p:- [1 3 5 7 9].

7. np.eye or np.identity:- Create a 2D array with ones on the diagonal & zeros elsewhere (Identity matrix).

eg:- c = np.eye(3).
Print(c).

o/p:- [[1. 0. 0.]
[0. 1. 0.]
[0. 0. 1.]]

eg:- a=np.eye(3,5)

o/p:- [[1. 0. 0. 0. 0.]
[0. 1. 0. 0. 0.]
[0. 0. 1. 0. 0.]]

Datatypes in Numpy:-

i - integer.
b - boolean
str - string
f = float.
m - time delta.
M - datetime.
O - Object
u - unsigned integer.
c - Complex float.
U - Unicode string.
V - fixed chunk of memory for other type (void).

Numpy Random numbers.

1. np.random.rand:- generates array with random numbers that are Uniformly distribute b/w 0 and 1.

0 → min, 1 → max. (min, max)

eg:- a = np.random.rand(5).
↳ How many numbers to generate.

a = np.random.rand(10,2)
↳ r, c.

2. np.random.randn – it helps to generate the random numbers normal distribution, μ(mean)=0 & (σ) stdev=1.
(these 2 parameters.)

eg:- a = np.random.randn(5).
↳ no's to generate.

b = np.random.rand(6,5)
↳ r, c.

→ No need to give range in randn because it gives b/w mean & stdev +ve or -ve.

3. np.random.randint – it helps to generate integers in a given range.
Uniform distribution.

eg:- a = np.random.randint(10) or (10, size=(5,4)).

randint(10, 40, size=(5, 10)).
↓ start ↓ end-1 ↓ r ↓ c

4. np.random.uniform – generates the float numbers.

eg:- a=np.random.uniform(10).

o/p:- 8.2081...

(or)

a = np.random.uniform(10, 20, size=(2,3)).

o/p:- [[. .]
[ ]].

Indexing, Slicing and Updating:-

→ Numpy arrays are mutable [i.e: Data stored in numpy array can be updated/changed]

Indexing:- Indexing in NumPy arrays works similarly to indexing in Python lists, but with additional capabilities for multi-dimensional arrays.

eg:- Single element indexing.

a=np.array([1,2,3,4,5])

Print(a[0]). o/p:- 1.

Print(a[5]) o/p:- Index Error.

eg:- Multidimensional indexing.

eg:- a=np.array([[1,2,3],[4,5,6]]).

print(a[1,2]) or print(a[1][2]) → cols.

o/p:- 6.

[[1, 2, 3] → 0
[4, 5, [6]] → 1 } rows
(r, c) → (1, 2).

Syntax:- arr[row_idx][col_idx] → row index → col index

(or)

arr[[row_idx],[col_idx]]. → Indexing starts from 0.

Negative indexing:- eg:- a=np.array([1,2,3,4])

Print(a[-1]) o/p:- 4.

Slicing:- Slicing allows you to access a portion of the array by specifying a range of indices. slicing in numpy array follows

Syntax. → arr[start : end : step].

if step size is positive → start should be on left
end should be on right.

if step size is -ve → start right
End - left.

1D array slicing → a = np.array([1,2,3,4])
Print(a[1:3]). o/p:- [2 3].

Multi-dimensional slicing →

```
      0  1  2  3
[[1  2  3  4] 0
 [5  6  7  8] 1
 [9  10 11 12]] 2
```

row col

eg:- arr[0:2 , 1:3].

start 0 start 1.
end 2-1=1 end 3-1=2
step 1 step 1.

$= \begin{bmatrix} 2 & 3 \\ 6 & 7 \end{bmatrix}$.

eg:- arr[1:2 , :].

o/p:- [5 6 7 8].

eg:- arr=np.array([[1,2,3,4], [5,6,7,8], [9,10,11,12]])

Print(arr[:, :: 2]) → Returns. Steps Column.

o/p:- [1 3
5 7
9 11].

Print(arr[::2, :]) → [[1, 2, 3, 4]
[9, 10 11 12]]

Print(arr[::2, ::2]) → [[1 3
9 11]]

Step slicing :- eg:- a = np.array([1,2,3])
print(a[::2]) o/p: [1 3]

Updating:- Update elements in a Numpy array using indexing or slicing just like accessing them.

eg:- a = np.array([1,2,3,4]).
a[1] = 10
Print(a) o/p:- [1 10 3 4].

→ a[2:4] = [20, 30].
print(a) o/p:- [1 10 20 30].

→ a = np.array([[1,2,3], [4,5,6]]).
a[0,1] = 10.
Print(a).

[[1 2 3] 0
[4 5 6]] 1.

2 → in place of 2 we are updating 10.

Flatten & Ravel

flatten( ):- Returns a Copy 'flatten( )' always returns a new array, which is flattened version of the Original array. Modifying the flattened array does not affect the array i.e; Original.

It is a method of ndarray object.

Ravel( ):- Returns a view (if Possible) 'ravel( )' returns a flattened array. If possible, it returns a view of the original array, which means that changes to the returned array may affect the Original array. If it cannot return a view (eg, if the array is not Contiguous in memory), it will return a Copy.

'ravel( )' is a function in the NumPy module.

np.ravel( )
np.flatten( ) → [ ] → [ ]
(5,10) (50,)

both are similar flatten & ravel
↓ Xerox of Original data
↓ Update the Original Copy

→ Ravel is faster than flatten( ) as it does not occupy any memory. Ravel returns a view of the Original array.

np.reshape( ):- is used to change the shape of an array without changing its data. The new shape must be compatible with the Original shape, means that the total no.of elements must remain the Same.

Syntax:- np.reshape(array,(r,c)).

→ If we don't give Correct rows&Cols it will give us "Value Error".

Iterating Over Numpy Array.

For 1 dim Array:-

a=np.array([1,2,3,4]). → it will pickup each value from the Collection until the Collection is exhausted.

for element in a:
    Print(element) o/p:- [1 2 3 4].

For 2dim Array:-

```
for row in arr2d:
    for item in arr2d.ravel():
        Print(item,end=" ")
```

Using np.nditer():- n-dimensional iteration

np.nditer() — is like flattening & raveling.

for readwrite np.nditer(arr, op_flags=['readwrite']):

↓

Operational flag.

Whenever we apply any Python operation on a numpy array, that operation will be BROADCASTED to each element in the array.

↓ It is also called as

Vectorization.

Python Operators on Numpy Array:- (+, -, *, %, <=, >=, ==)

→ It will perform the operation on each element at a time of a numpy array.

```
x = np.array([[1,2,3],[4,5,6]])
Print(x+5)
o/p:- [[6,7,8]
       [9,10,11]]
```

→ % = remainder

x>=3, True or False (Boolean).

Print(...) → Ellipsis — An inbuilt Constant in python

→ It Can do the pass keyword in python.

Numpy Maths :-

np.sqrt(arr) → Square Root.

np.exp(arr) → Exponent.

np.sin(0) → Trigonetic Sin.

np.cos(0) → " ... Cos.

$x = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}; y = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$.

$x+y = \begin{bmatrix} 6 & 8 \\ 10 & 12 \end{bmatrix}$.

np.add(x,y). → for addition of two Arrays

np.subtract(x,y)

np.multiply(x,y).

np.divide(x,y).

Matrix Multiplication:-

$x = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}; y = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$.

$x*y = \begin{bmatrix} 1\times5+2\times7 & 1\times6+2\times8 \\ 3\times5+4\times7 & 3\times6+4\times8 \end{bmatrix} = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}$.

np.matmul(x,y).
np.dot(x,y)
(x@y)
} — 3 Ways for matrix Multiplication.

np.diag(x) → for diagonal elements.
where row id & col id are same.

np.T → Transpose (rows change into Columns & Columns changes to rows)

Numpy Statistics

np.min() → Minimum

np.max() → Maximum.

np.mean() → Mean

np.median() → Median.

np.var() → Variance.

np.std() → std dev, np.corrcoef() → Correlate Coefficients.

$\begin{array}{c|cc} & \text{Col } 1 & 2 \\ \hline \text{Row } 1 & 1 & 2 \\ 2 & 3 & 4 \end{array} = [10]$.

Col wise sum = np.sum(x, axis=0) = [4 6].

Row wise sum = np.sum(x, axis=1) = [3 7].

[↓↓↓] → axis=0 → Col wise.

[⇉] → axis=1 → row wise

min
max
mean
Var
median
std
→ Univariate.
[Passing Single Variable]

Corrcoefficient, pearson
Bivariate Function.

Miscellaneous Topics:-

Linspace:- If we want to Create elementswith proper spacing inside numpy array (at equal distance).

Syntax:- np.linspace(begin value, end value, How many element)

eg:- np.linspace(1, 5, 9).

Sorting:- By default it takes rowwise sorting ie; axis=1.

np.sort(axis=0) i.e; Col wise.

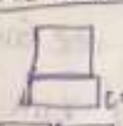
np.sort(axis=1) i.e; row wise.

Stacking:- Joining arrays along a new axis.

np.Vstack:- Stacks arrays in sequence vertically (row wise).

np.Hstack:- Stacks arrays in sequence horizontally (column wise).

np.stack :- Joins a sequence of arrays along a new axis.

np.vstack([arr1, arr2]) → Vertically  i.e. for vertical stacking.

np.hstack([arr1, arr2]) → horizontal i.e for horizontal stacking.

Concatenate:- The Concatenate function joins a sequence of arrays along an existing axis.

np. Concatenate ([arr1, arr2], axis=0) for Vertical stacking

np. Concatenate([arr1, arr2], axis=1) for horizonal stacking

Append:-

np.append([1,2], axis=0) → Vertical stacking

np. append([1,2], axis=1) → horizontal stacking.

Adds Values to the end of an array, optionally along a specified axis.

Where:- where() processing Array Elements Conditionally.

np. where(arr > 64, 0, 1).

i.e; If value is > 64 Convert to 0, Otherwise Convert to 1

np.where(Condition, Expression_1, Expression_2).
↓ True Condition ↓ False Condition.

Argsort:-

argmin — np.argmin( ) → minimum value index number.
argmax — np.argmax( ) → maximum value index number.
argsort — indexes are sorted.

[10, -5, 6, -1].
= [1, 3, 2, 0].

eg:- arr = np.array([10, 25, 3, 17])
min_index = np.argmin(arr) , max_index = np.argmax(arr)
Print(min_index) → o/p:-2
Print(arr[min_index]) → o/p:3
Print(max_index) → o/p:-1. [25].

eg:- m = np.array([[1, 7, 3],
[4, 2, 9]])
Print(np.argmin(m)) → o/p:0
Print(np.argmax(m)) → o/p:-5

Explanation:- The array is flattened to [1, 7, 3, 4, 2, 9].
min value is 1 at index 0.
Max value is 9 at index 5.

→ Using axis:- axis=0 Col-wise.
Print(np.argmin(m, axis=0)) → o/p:- [0 1 0].
1st Col → 1 v/s 4 → min at row 0.
2nd Col → 7 v/s 2 → at row 1.
3rd Col → 3 v/s 9 → min at row 0.

Print(np.argmax(m, axis=1)) → o/p [1 2].